0329. Longest Increasing Path in a Matrix

2-D Dynamic Programming | LeetCode: https://leetcode.com/problems/longest-in

Problem

- Return the length of the longest strictly increasing path moving up/down/left/right.

Intuition

- Strictly increasing paths can never revisit a cell, so the "graph" is a DAG and memoisation is safe.
- Best(r, c) = 1 + max best(neighbour) over neighbours with a larger value.

Brute Force

- DFS from every cell without memo: exponential.

Optimal Approach

- Memo[r][c] = 0 (unknown).
- Dfs(r, c): if memo set, return it; else compute from larger neighbours and store.
- Answer = max dfs over all cells.

Worked Example

```
[[9,9,4],[6,6,8],[2,1,1]].

path 1 -> 2 -> 6 -> 9 : length 4
memo(9)=1, memo(6)=2, memo(2)=3, memo(1 at (2,1))=4
answer 4
```

Java Solution

```java
public class LongestIncreasingPathInMatrix {
    private static final int[][] DIRS = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};

    public int longestIncreasingPath(int[][] matrix) {
        int m = matrix.length, n = matrix[0].length, best = 0;
        int[][] memo = new int[m][n];
        for (int r = 0; r < m; r++)
            for (int c = 0; c < n; c++) best = Math.max(best, dfs(matrix, r, c, memo));
        return best;
    }

    private int dfs(int[][] g, int r, int c, int[][] memo) {
        if (memo[r][c] != 0) return memo[r][c];
        int best = 1;
        for (int[] d : DIRS) {
            int nr = r + d[0], nc = c + d[1];
            if (nr >= 0 && nc >= 0 && nr < g.length && nc < g[0].length && g[nr][nc] > g[r][c])
                best = Math.max(best, 1 + dfs(g, nr, nc, memo));
        }
        return memo[r][c] = best;
    }
}
```

Complexity

- Time: O(m n). Space: O(m n).

Pitfalls

- No visited set needed - strict increase prevents cycles.